# FONCTIONS D'UNE VARIABLE RÉELLE

## 1. Les fonctions affines.

**a.** Rappeler la forme sous laquelle peut s'écrire une fonction affine.
La forme sous la forme peut s'écrire sous cette forme:
**Une fonction affine s'écrit sous la forme : f(x) = ax+b**

**b.** Rappeler à quoi correspondent les paramètres utilisés dans cette forme.
**«a» veut dire coefficient directeur et «b» c'est l'ordonnée à l'origine.**

Dans un repère orthonormé, on considère deux points A et B de coordonnées:

A (1; – 3) et B (4; 3).

Déterminer une équation de la fonction affine dont la courbe représentative passe par les points A et B.

**f(x) = 2x-5**
**pour trouver «a» par le calcul :** $a = (y_b-y_a)/(x_b-x_a)$
$= (3-(-3)/4-1$
$= 6/3$
$= 2$

**Pour trouver «b» par le calcul :**
**A appartient a la droite → ya= a*xa+b → -3 = 2*1 + b → b = -3-2 = -5**

**c.** De manière générale, on considère désormais deux points A et B dans un repère orthonormé dont on note les coordonnées de la manière suivante :

A $(x_A ; y_A)$ et B $(x_B ; y_B)$.

Déterminer une équation de la fonction affine dont la courbe représentative passe par les points A et B en fonction des coordonnées des points A et B.

**Pour trouver «a» par le calcul: $a = (y_b-y_a) / (x_b-x_a)$**
**Pour trouver «b» par le calcul: $y_a = a*x_a + b \rightarrow b = y_a-a*x_a$**
**d'où : $f(x) = ((y_b-y_a) / (x_b-x_a))x + (y_a-(y_b-y_a) / (x_b-x_a)x_a)$**

**d.** Proposer un algorithme permettant de déterminer une équation d'une fonction affine en fonction des coordonnées de deux points A et B saisies par l'utilisateur.

```
Saisir x1, y1
  Saisir x2, y2

  si x1 = x2 alors
        afficher "Les points ont la même abscisse, donc la droite est verticale."
  sinon
    a <- (y2 - y1) / (x2 - x1)
    b <- y1 - a * x1
    afficher "L'équation de la droite est : y = ", a, "x + ", b
  fin si

FIN
```

**e.** Traduire cet algorithme en langage Python.

## Programme :

```
x1 = float(input("x1 = "))
y1 = float(input("y1 = "))
x2 = float(input("x2 = "))
y2 = float(input("y2 = "))

if x1 == x2:
  print("Les points ont la même abscisse, donc la droite est verticale.")
else:
  a = (y2 - y1) / (x2 - x1)
  b = y1 - a * x1
  print("L'équation de la droite est : y =", a, "x +", b)
```

## Exemple d'utilisation :

```
x1 = 1
y1 = 3
x2 = 3
y2 = 7
```

### Résultat :

```
L'équation de la droite est : y = 2.0 x + 1.0
```

**f.** Créer une fonction *eq_fonction_affine (xA,yA,xB,yB)* prenant en paramètres les coordonnées des points A et B et retournant l'expression de la fonction affine correspondante.

### Programme :

```
def eq_fonction_affine(xA, yA, xB, yB):
  if xA == xB:
    return "La droite est verticale : x = " + str(xA)



  a = (yB - yA) / (xB - xA)
  b = yA - a * xA



  return f"y = {a}x + {b}"
```

### Exemple d'utilisation :

```
print(eq_fonction_affine(1, 3, 3, 7))
```

### Résultat :

```
y = 2.0x + 1.0
```

## 2. Résolution d'une équation du second degré.

**a.** Rappeler la forme sous laquelle peut s'écrire une équation du second degré.

**Une équation du second degré s'écrit sous la forme : $ax^2 + bx + c$.**

**b.** Rappeler les différentes solutions possibles d'une telle équation.

**On calcule, tout d'abord, le discriminant «delta» = $b^2$ - 4 * a * c :**

**Si delta > 0 : L'équation possède 2 solutions réelles distinctes :**

$x_1$ = (-b - sqrt(delta)) / (2 * a)

$x_2$ = (-b + sqrt(delta)) / (2 * a)

**Si delta = 0 : L'équation possède 1 solution réelle unique (double):**

$x_0$ = -b / (2 * a)

**Si delta < 0 : L'équation n'a aucune solution réelle.**

**c.** Proposer un algorithme permettant de déterminer les solutions d'une équation du second degré en fonction des coefficients saisis par l'utilisateur.

Saisir a, b, c

```
a <- nombre saisi
b <- nombre saisi
c <- nombre saisi

si a = 0 alors
   afficher "Ce n'est pas une équation du second degré"
sinon
   delta <- b² - 4 * a * c

   si delta > 0 alors
      x1 <- (-b - racine(delta)) / (2 * a)
      x2 <- (-b + racine(delta)) / (2 * a)
      afficher "2 solutions :", x1, x2
   sinon si delta = 0 alors
      x0 <- -b / (2 * a)
      afficher "1 solution :", x0
   sinon
      afficher "Aucune solution réelle"
   fin si
fin si

fin
```

**d.** Traduire cet algorithme en langage Python.

### Programme :

```
a = float(input("Entrez a : "))
b = float(input("Entrez b : "))
c = float(input("Entrez c : "))

if a == 0:
  print("Erreur : Ce n'est pas une équation du second degré")
else:
  delta = b**2 - 4 * a * c

  if delta > 0:
    x1 = (-b - math.sqrt(delta)) / (2 * a)
    x2 = (-b + math.sqrt(delta)) / (2 * a)
    print("Il y a 2 solutions réelles :", x1, "et", x2)
  elif delta == 0:
    x0 = -b / (2 * a)
    print("Il y a 1 solution unique :", x0)
  else:
    print("Il n'y a aucune solution réelle")
```

### Exemple d'utilisation :

```
a = 1
b = 2
c = -3
```

### Résultat :

```
2 solutions : -3.0 1.0
```

**e.** Créer une fonction *resol_eq_second_degre (a,b,c)* prenant en paramètres les coefficients de l'équation du second degré et retournant le nombre de solutions ainsi que leur valeur.

### Programme :

```
def resol_eq_second_degre(a, b, c):
  if a == 0:
    return "erreur", "Ce n'est pas une équation du second degré"

  delta = b**2 - 4 * a * c
```

```
  if delta > 0:
    x1 = (-b - math.sqrt(delta)) / (2 * a)
    x2 = (-b + math.sqrt(delta)) / (2 * a)
    return 2, [x1, x2]

  elif delta == 0:
    x0 = -b / (2 * a)
    return 1, [x0]

  else:
    return 0, []
```

## Exemple d'utilisation :

```
nb, solutions = resol_eq_second_degre(1, 2, -3)
print(nb)
print(solutions)
```

## Résultat :

```
2
[-3.0, 1.0]
```